Taurino | Git & GitHub Workflow

PowerShell-Handbuch für Änderungen, Pull Requests, CI, Releases und PyPI · 21. Oktober 2026

Zweck

Diese Anleitung beschreibt den manuellen Standardprozess für neue Änderungen im Repository taurino-foundation/taurino. Sie richtet sich nach den bereitgestellten GitHub-Actions-Dateien und respektiert den geschützten Branch main. Der automatische PowerShell-Gesamtablauf ist bewusst nicht enthalten.

1. Commit- und Pull-Request-Regeln

Änderung	Beispieltitel	Typische Release-Folge
Neue Funktion	feat(window): add fullscreen support	Minor
Fehlerbehebung	fix(ipc): prevent connection timeout	Patch
Performance	perf(runtime): reduce startup overhead	Konfigurationsabhängig
Refactoring	refactor(core): simplify event handling	Normalerweise keine
Dokumentation	docs: update README	Normalerweise keine
Tests	test(runtime): add shutdown tests	Normalerweise keine
CI / Workflows	ci: improve Windows checks	Normalerweise keine
Build-System	build: update maturin config	Normalerweise keine
Wartung	chore: update dependencies	Normalerweise keine
Rücknahme	revert: revert window changes	Konfigurationsabhängig
Breaking Change	feat(api)!: redesign window builder	Gemäß Release-Konfiguration

Der Workflow pr-title.yml prüft ausdrücklich den Pull-Request-Titel anhand des Conventional-Commit-Musters. Für Commit-Nachrichten wird dieselbe Schreibweise empfohlen. Tatsächliche Release-Versionen hängen von der release-please-Konfiguration und der bisherigen Versionshistorie ab.

2. Standardablauf für jede neue Änderung

Reihenfolge: Branch erstellen → Code ändern → lokal prüfen → committen → pushen → Pull Request → CI/Reviews → Squash-Merge → main aktualisieren.

Schritt A – Branch erstellen

```
git switch main
git pull --ff-only origin main
git switch -c feat/window-fullscreen
```

Erstelle den Branch bevor du mit der neuen Änderung beginnst. Arbeite nicht direkt auf main. Bei vorhandenen lokalen Änderungen vorher git status prüfen.

Schritt B – Änderungen lokal prüfen und committen

```
git status
cargo fmt --all
cargo check --workspace --all-targets
git add -A
git diff --cached --stat
git commit -m "feat(window): add fullscreen support"
```

Mit git add -A werden auch Löschungen übernommen. Darum die Änderungen vor jedem Commit prüfen. Je nach Änderung zusätzlich Tests, Clippy und Python-Prüfungen ausführen.

Schritt C – Branch pushen und Pull Request eröffnen

```
git push -u origin feat/window-fullscreen

gh pr create `
  --base main `
  --head feat/window-fullscreen `
  --title "feat(window): add fullscreen support" `
  --body "Implement fullscreen support for desktop windows."
```

Schritt D – GitHub Actions prüfen und zusammenführen

```
gh pr status
gh pr checks --watch
gh pr view --web

# Nach erfolgreichen Pflichtprüfungen und Reviews:
gh pr merge --squash --delete-branch
```

Die CI prüft Rust auf Linux, Windows und macOS sowie Python auf mehreren Plattformen und Versionen; zusätzlich laufen Linting, Metadaten- und gegebenenfalls Sicherheitsprüfungen. Bei fehlgeschlagenen Pflichtprüfungen nicht mergen.

Schritt E – lokalen main aktualisieren

```
git switch main
git pull --ff-only origin main
git branch --delete feat/window-fullscreen
```

3. Release Management und PyPI

GitHub Release: Durch einen Merge in main startet der Workflow Release Management. release-please kann abhängig von Conventional Commits und Konfiguration einen Release-PR erstellen. Nach dessen Merge kann daraus eine GitHub Release mit Tag entstehen.

PyPI: Laut bereitgestellter publish-python.yaml erfolgt keine automatische Veröffentlichung durch Push, Tag oder GitHub Release. Sie wird ausschließlich manuell über workflow_dispatch mit dem bereits veröffentlichten Release-Tag gestartet.

Beispiel: Version v0.1.1 bewusst veröffentlichen

```
gh release view v0.1.1

gh workflow run publish-python.yaml `
  --ref main `
  -f release_tag=v0.1.1

gh run list --workflow publish-python.yaml
```

Dabei baut der Workflow Wheels und ein Source-Archiv und veröffentlicht mit PyPI Trusted Publishing (OIDC). Falls die GitHub-Umgebung pypi Genehmiger voraussetzt, muss zusätzlich diese Freigabe erfolgen.

4. Kurzreferenz: Git und gh

Aufgabe	Befehl
Branch erstellen	git switch -c feat/name
Änderungen prüfen	git status
Alle Änderungen stagen	git add -A
Commit erstellen	git commit -m "feat: add feature"
Branch hochladen	git push -u origin HEAD
Pull Request erstellen	gh pr create --base main --fill
PR-Checks überwachen	gh pr checks --watch
Squash-Merge	gh pr merge --squash --delete-branch
Main aktualisieren	git pull --ff-only origin main
Offene Release-PRs prüfen	gh pr list --state open
Releases anzeigen	gh release list
Workflow-Ausführungen	gh run list

5. Zusätzliche technische Hinweise

Cargo.lock: Die CI führt cargo update --workspace aus, stellt danach aber nicht ausdrücklich sicher, dass die Datei unverändert geblieben ist. Für reproduzierbare Ergebnisse sollte ein zusätzlicher Lockfile-Abgleich in der CI erwogen werden.

Publish-Freigabe: Das gezeigte Publish-Skript kontrolliert Release-Tag-Format und Veröffentlichungsstatus. Eine explizite Prüfung, dass der Release-Commit zur erwarteten main-Historie gehört, ist zusätzlich sinnvoll.

Versionskonsistenz: Die bereitgestellten Skripte check_metadata.py und verify_tag.py überprüfen unter anderem Cargo-/Workspace-Version, Maturin-Pfad, ABI3-Konfiguration und die Übereinstimmung des Release-Tags.

Merksatz: Für reguläre Änderungen immer Feature-Branch → PR → Checks → Merge. Eine PyPI-Veröffentlichung ist ein separater, bewusster Schritt.